Today's Topic

⇒ Job creation using fork, pthread_create

⇒ Process vs thread

⇒ Address Space sharing and Need of IPC.

⇒ callback code. in practical use.

In Linux

using $\boxed{\text{fork()}}$ System call

→ To create a new job (without loading an executable)
process

→ fork creates duplicate of the current PCB.

→ If the new job (child job) requires to launch/load an executable then exec-family of function can be used.

→ The parent and child have different address space to work with that means the Page Map of parent and child ~~may~~ differs

→ e.g. A global variable (visible to par & child), if modified exclusively in child, then parent's copy is not affected.

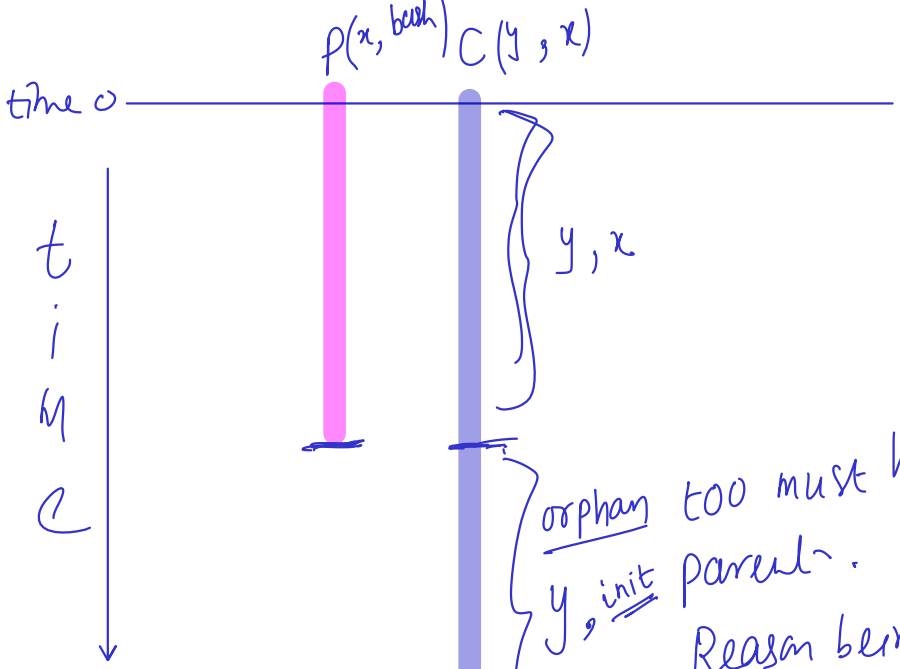
→ If the child's exclusive data in needed in the parent or vice-versa. or (If one process's data is needed in another).

What to do?

⇒ Use IPC (Inter Process Communication) events.

↳ pipe, fifo, Msg Ques, Shared memory, socket

→ child (fork created child) are background jobs.

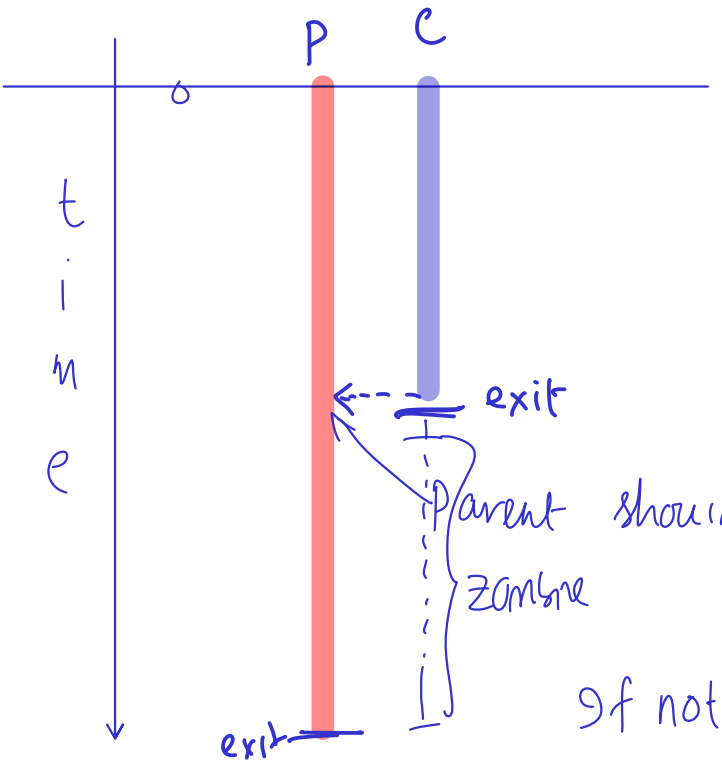


init / systemd } pid: 1

orphan too must have a parent. y, init

Reason being:

whenever a job exits, the exit value MUST be collected by parent.



Parent should collect the exit value.

↳ wait(), waitpid()

If not collected, then child becomes ZOMBIE (Dead, not cleaned up)

child job either becomes zombie or orphan.